CHƯƠNG 3 · BÀI 3

3.3. Module & Fetch API

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

> **Mục tiêu bài học**
>
> - Chia mã thành các module bằng import và export
> - Gọi API bằng fetch và xử lý dữ liệu JSON
> - Xử lý lỗi và trạng thái khi gọi API

1. Module

Module cho phép chia chương trình thành nhiều file nhỏ. Dùng `export` để chia sẻ hàm, biến ra ngoài và `import` để sử dụng. Khi dùng trên trình duyệt phải khai báo `type="module"` ở thẻ script.

```
// toan.js
export function cong(a, b) { return a + b; }
export default function nhan(a, b) { return a * b; }

// main.js
import nhan, { cong } from "./toan.js";
```

2. Fetch API

Hàm `fetch(url)` gửi yêu cầu HTTP và trả về Promise. Kết quả là đối tượng Response; gọi `.json()` để đọc nội dung JSON. Chú ý `fetch` chỉ báo lỗi khi mất kết nối, còn mã trạng thái 404 hoặc 500 vẫn là thành công về mặt mạng nên cần kiểm tra `res.ok`.

```
async function layKhoaHoc() {
  const res = await fetch("/api/courses");
  if (!res.ok) throw new Error("Lỗi " + res.status);
  return res.json();
}
```

3. Gửi dữ liệu

Để gửi dữ liệu lên máy chủ, đặt `method: "POST"`, khai báo header `Content-Type: application/json` và truyền nội dung bằng `body: JSON.stringify(dulieu)`.

```
await fetch("/api/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email, password }),
});
```

4. Xử lý lỗi và trạng thái

Khi gọi API nên quản lý ba trạng thái: đang tải, thành công và lỗi; bọc lời gọi trong `try...catch` và hiển thị thông báo thân thiện cho người dùng.

Tóm tắt

- `export` / `import` chia mã thành module; thẻ script cần `type="module"`.
- `fetch` trả về Promise; cần kiểm tra `res.ok` và gọi `.json()`.
- Gửi dữ liệu dùng `POST` với header `Content-Type` và `body` dạng JSON.

Câu hỏi ôn tập và bài tập vận dụng

1. Tách các hàm tính toán ra một module riêng và import vào file chính.
2. Gọi một API công khai bằng fetch và hiển thị danh sách kết quả lên trang.
3. Thêm hiển thị trạng thái đang tải và thông báo khi gọi API thất bại.

CHƯƠNG HỌC THÊM · BÀI TỔNG HỢP

Học thêm.Tổng hợp. Học thêm, bài tập và hướng dẫn giải

Tài liệu học tập JavaScript Từ Zero Đến Hero · Nội dung biên soạn trong dự án StudyOnline.

JavaScript — Học thêm về dữ liệu, DOM và bất đồng bộ

Tài liệu bổ sung cho khóa JavaScript từ nền tảng tới Fetch API. Ví dụ chạy trong trình duyệt; phần module nên chạy qua máy chủ phát triển thay vì mở file trực tiếp.

> ### Mục tiêu học thêm
>
> - Phân biệt chuyển kiểu và so sánh nghiêm ngặt
> - Xây dựng DOM từ dữ liệu thay vì dùng HTML không tin cậy
> - Xử lý lỗi HTTP và tình huống phản hồi API đến sai thứ tự

1. Chuyển kiểu và tham chiếu

Chuỗi nhập từ form vẫn là chuỗi. Number('') trả 0 nên phải kiểm tra chuỗi trắng trước khi chuyển. Sau chuyển, dùng Number.isFinite để loại NaN và vô cực khi yêu cầu một số hữu hạn.

```
function readScore(raw) {
  if (raw.trim() === '') throw new Error('Chưa nhập điểm');
  const score = Number(raw);
  if (!Number.isFinite(score) || score < 0 || score > 10) {
    throw new Error('Điểm không hợp lệ');
  }
  return score;
}
console.log(readScore('7.5')); // 7.5
console.log('5' === 5); // false
```

const ngăn gán lại tên biến, không đóng băng đối tượng. Một mảng khai báo const vẫn có thể push. Spread [...items] tạo mảng mới nhưng chỉ sao chép nông; đối tượng bên trong có thể vẫn dùng chung.

2. DOM và sự kiện

Với To-do List, dữ liệu là nguồn chính; DOM là kết quả hiển thị. Mỗi công việc cần mã ổn định để xóa đúng dù danh sách bị sắp xếp. Dùng textContent cho nội dung người dùng nhập, tránh chèn thẳng vào innerHTML.

```
function renderTask(task) {
  const li = document.createElement('li');
  const label = document.createElement('span');
  label.textContent = task.title;
  const button = document.createElement('button');
  button.type = 'button';
  button.textContent = 'Xóa';
  button.addEventListener('click', () => removeTask(task.id));
  li.append(label, button);
  return li;
}
```

removeTask là hàm của ứng dụng: lọc mảng theo id, lưu lại rồi render. Khi xử lý submit, gọi preventDefault nếu form được quản lý bằng JavaScript; kiểm tra title.trim() trước khi thêm. Không gắn lại cùng một listener mỗi lần render trên cùng nút.

3. Bất đồng bộ và thứ tự thực thi

await tạm dừng phần tiếp theo của hàm async, không khóa toàn bộ trang. Promise thất bại cần catch hoặc try/catch. Khi hai tác vụ độc lập có thể chạy cùng lúc, Promise.all giảm thời gian chờ nhưng sẽ reject nếu một promise reject.

```
console.log('A');
Promise.resolve().then(() => console.log('B'));
console.log('C');
// A, C, B
```

Trong ví dụ, callback then được chạy sau phần đồng bộ hiện tại. Hiểu thứ tự này giúp tránh đọc dữ liệu trước khi request hoàn tất.

4. Fetch và phản hồi cũ

fetch không tự ném lỗi chỉ vì server trả 404 hoặc 500. Cần kiểm tra response.ok trước khi đọc dữ liệu dùng cho giao diện.

```
let controller;
async function searchCourses(term) {
  controller?.abort();
  controller = new AbortController();
  const signal = controller.signal;
  try {
    const response = await fetch('/api/courses?q=' + encodeURIComponent(term), { signal
});
    if (!response.ok) throw new Error('HTTP ' + response.status);
    const data = await response.json();
    if (!signal.aborted) console.log(data);
  } catch (error) {
    if (error.name !== 'AbortError') console.error(error.message);
  }
}
```

Đường dẫn API là ví dụ, cần thay theo backend đang dùng. Hủy request cũ giúp kết quả tìm kiếm trước không ghi đè kết quả sau. Ứng dụng thực tế nên hiện trạng thái tải, rỗng và lỗi thay vì chỉ console.log.

Câu hỏi ôn tập và bài tập vận dụng

1. Dễ: dự đoán typeof của dữ liệu lấy từ input; giải thích vì sao '2' + '3' là '23'. Đáp án: dữ liệu là chuỗi, toán tử cộng nối chuỗi khi các toán hạng là chuỗi.
2. Trung bình: thêm lọc việc đã hoàn thành vào To-do List nhưng không mất dữ liệu gốc. Giữ toàn bộ tasks, tạo visibleTasks bằng filter theo trạng thái; không thay tasks bằng danh sách đã lọc.
3. Khó: tìm kiếm khi người dùng nhập liên tiếp ba từ khóa, phản hồi lần đầu về cuối. Dùng AbortController hoặc số phiên request để chỉ nhận kết quả mới nhất; thử mạng chậm và lỗi 500.

Dự án mở rộng và tiêu chí hoàn thành

Thêm lưu localStorage cho To-do List. Khi đọc JSON, bắt lỗi parse và kiểm tra dữ liệu có đúng mảng công việc không. Thử tiêu đề rỗng, chuỗi chứa thẻ HTML, mã trùng và tải lại trang. Bài đạt khi thao tác thêm/xóa/lọc nhất quán, nội dung người dùng được hiển thị như văn bản và lỗi lưu trữ không khiến trang trắng.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises

https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Working_with_objects

Module & Fetch API — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: fetch trả 404 nhưng catch không chạy. fetch không tự coi HTTP 4xx/5xx là Promise rejection; phải kiểm tra response.ok. JSON lỗi cũng cần được xử lý.

```
export async function getCourses(url, signal) {
  const response = await fetch(url, {signal});
  if (!response.ok) throw new Error(`HTTP ${response.status}`);
  const body = await response.json();
  if (!Array.isArray(body.data)) throw new Error("Sai cấu trúc dữ liệu");
  return body.data;
}
```

Tự luyện: thử 200 có dữ liệu, 200 rỗng, 404, JSON hỏng và hủy request. **Hướng giải:** mỗi trạng thái có cách hiển thị rõ; AbortController không đồng nghĩa backend hoàn tác hành động đã nhận. Module trình duyệt cần `type="module"`; lỗi CORS phải sửa cấu hình đúng phía server, không tắt bảo vệ trình duyệt.

Nguồn học tiếp

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide